# İleri Python Programlama – Hafta 5 Lab Ödevi

## Dekoratörler

**Ders:** İleri Python Programlama · **Kurum:** İstanbul Nişantaşı Üniversitesi · **Öğretim üyesi:** Dr. Öğr. Üyesi Murat Emeç · **Dönem:** 2026–2027 Güz

**Öğrenci:** _____ **Numara:** _____ **GitHub deposu:** _____

Bu lab, uzay projesine kendi dekoratörlerinizi ve dekoratörlerle çalışan bir komut sistemi ekler. Hafta 4'ün son commit'inden devam edin. Dekoratörleri `uzay_oyunu/dekoratorler.py`, komutları `uzay_oyunu/komutlar.py` modülüne yazın. Her dekoratör `functools.wraps` kullanmalı ve `tests/test_dekoratorler.py` içinde test edilmelidir.

### Görev 1 – Süre ölçen dekoratör (15 puan)

`sure_olc` dekoratörünü yazın: fonksiyonun çalışma süresini `time.perf_counter()` ile ölçüp "`ad: 0.0123 sn`" biçiminde yazdırsın ve fonksiyonun sonucunu döndürsün. Üç test yazın: sonuç değişmeden dönüyor mu; adlı argümanlarla çağrı çalışıyor mu; `__name__` ve `__doc__` korunuyor mu. Hafta 4'teki önbelleksiz rota sayacına uygulayıp `adim=20` için süreyi gösterin.

### Görev 2 – Çağrı sayacı ve dekoratör sırası (20 puan)

Çağrıları fonksiyonun bir niteliğinde sayan `say` dekoratörünü yazın. Ardından özyinelemeli Fibonacci'yi üç biçimde tanımlayıp `fib(20)` için çağrı sayısını ölçün: yalnızca `@say`; `@say` üstte, `@lru_cache` altta; `@lru_cache` üstte, `@say` altta. Üç sonucu ve **neden farklı olduklarını** `dekorator_sirasi.md` dosyasında açıklayın. (İpucu: üçüncü durumda sayaca `fib.__wrapped__.cagri` ile ulaşılır.)

```
Yalnızca @say            : 21891
@say üstte, @lru_cache altta: 39
@lru_cache üstte, @say altta: 21
```

### Görev 3 – Loglayan dekoratör (15 puan)

`log_cagri` dekoratörü, her çağrıdan önce fonksiyon adını ve argümanlarını (konumsal ve adlı), sonra da dönüş değerini yazdırsın. `galaksi_kur` ve `hedef_menusu` fonksiyonlarına uygulayın; bu sırada testlerin bozulmadığını kontrol edin.

```
-> yolculuk('orion', hiz=3)
<- yolculuk = 'orion hiz=3'
```

### Görev 4 – Komut sistemi: kayıt deseni (30 puan)

`komutlar.py` içinde bir `KOMUTLAR` sözlüğü ve argüman alan bir `komut(ad, aciklama="")` dekoratörü yazın. Dekoratör fonksiyonu **değiştirmeden** `(fonksiyon, aciklama)` olarak kaydetsin; aynı ad iki kez kullanılırsa tanım anında `ValueError` fırlatsın. Oyunun durumunu (`konum`, `baglantilar`, `credits`, `engines`, `copilot`) taşıyan bir `durum` sözlüğü alan üç komut tanımlayın: `yardim`, `envanter`, `harita`. `travel()` içinde oyuncunun girdisi bir komut adıysa komutu çalıştırıp sonucu yazdırın ve **aynı menüyü tekrar gösterin**; değilse eskisi gibi gezegen seçimi olarak yorumlayın. Eski testler bozulmamalı; her komut için yeni bir test yazın.

```
Nereye gitmek istersin?
[1] orion
[2] dünya
[3] kara_delik
> yardim
envanter  eşyalarını gösterir
harita    gidebileceğin gezegenleri gösterir
yardim    komutları listeler
> harita
orion, dünya, kara_delik
```

## Görev 5 – Argüman alan dekoratör (20 puan)

`sinirla(alt, ust)` dekoratörünü yazın: dekore edilen fonksiyonun ilk argümanı `[alt, ust]` aralığında değilse anlamlı bir `ValueError` fırlatsın. `yakit_harca(miktar)` fonksiyonuna `@sinirla(0, 100)` uygulayın. Testlerde sınır değerlerini (0 ve 100) ve aralık dışı değerleri ayrı ayrı deneyin.

```
>>> yakit_harca(30)
70
>>> yakit_harca(150)
ValueError: yakit_harca: 150 değeri 0-100 aralığında değil
```

**Bonus (+10).** `functools.lru_cache` kullanmadan, closure içindeki bir sözlükle çalışan `hafiza(maxsize=None)` dekoratörünü yazın. `maxsize` verildiğinde önbellek dolunca **en eski** girdiyi silsin. Sonuçlarını `lru_cache` ile karşılaştıran bir test ekleyin.

## Teslim ve puanlama

- **Teslim:** GitHub deposunun bağlantısı, bir sonraki lab dersinden önce. Her görev en az bir commit.
- Tüm eski ve yeni testler `uv run pytest` ile geçmelidir.
- Kopya tespit edilen ödevler, veren ve alan için sıfır puan alır. Öğrenci her dekoratörün tanım ve çağrı anında ne yaptığını açıklayabilmelidir.

| Ölçüt | Pay | Tam puan için |
|---|---|---|
| Doğruluk | %50 | Dekoratörler ve komutlar örnek çıktıları veriyor, testler geçiyor |
| Dekoratör tasarımı | %20 | wraps, *args/**kwargs ve return eksiksiz; kayıt fonksiyonu değiştirmiyor |
| Testler | %15 | İsim/belge koruma, sınır değerleri ve komut testleri mevcut |
| Belgeleme | %15 | Docstring'ler ve dekorator_sirasi.md açıklaması |